RICE UNIVERSITY

Data Parallelism and Sharding

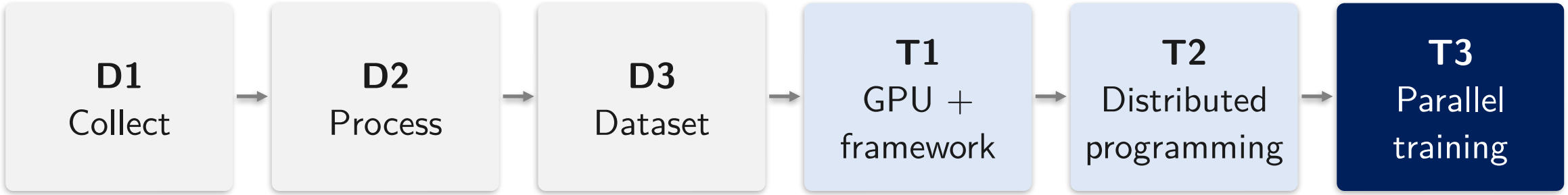
Many copies of one model, one averaged gradient, and how to stop paying for the copies

Yuke Wang

Rice University

COMP 329 · Spring 2027 · Week 12, Lecture 1

Where we are in the pipeline



In our pipeline

Today: train **one model on many GPUs** by splitting the batch. T3 is released after this lecture; **T2 is due Friday**.

Week 11: collectives and their cost → today: the one all-reduce a training step needs

Why train on many GPUs?

Train a 7.5-billion-parameter model on 2 trillion tokens.
You have **one H100 GPU**.

How long does training take?



Answer

One GPU is **far too slow**. We must split **one training run** across many GPUs — but how do we split it?

Source: Estimate: 6 FLOP/parameter/token (Kaplan et al. 2020); 2T tokens (Llama 2); H100 bf16 peak (datasheet) at an assumed 40%.

Three ways to split training

Data parallel

GPU 0



½ of batch

GPU 1



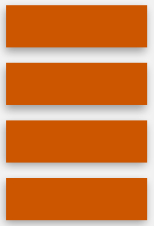
½ of batch

copy model, split batch

today

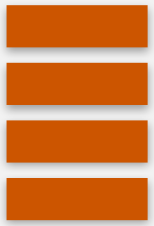
Tensor parallel

GPU 0



½ each layer

GPU 1



½ each layer

split every layer

Week 12, Lecture 2

Pipeline parallel

GPU 0



layers 1–2

GPU 1



layers 3–4

give each GPU some layers

Week 13

Key idea

Data parallelism comes first: the model is **copied, not cut**, so the model code does not change — only one averaged gradient per step.

Where data parallelism shows up

- **PyTorch**: **DDP** trains one model on many GPUs; **FSDP** also saves memory.
- **Large models**: the outer split, combined with tensor and pipeline parallelism.
- **Costs**: one all-reduce **per step**, and a full copy of the model on **every** GPU.

In our pipeline

T3, part 1: your data-parallel loop vs a single-process reference vs DDP, on D3 data. Lab 5: tensor and pipeline.

Why data parallelism?

One GPU would take **years**. Data parallelism is the first split: **the same model on more data per step**, with one averaged gradient.

Source: PyTorch documentation, torch.nn.parallel.DistributedDataParallel and "DDP notes"; Li et al., VLDB 2020.

Learning goals for today

- **Explain** why averaging the GPUs' gradients gives the full-batch gradient — and fix it when shards are unequal.
- **Estimate** the one all-reduce per step, and explain how **DDP** hides it behind backward.
- **Account** for the **16 bytes per parameter** that training with Adam needs.
- **Compute** per-GPU memory when **ZeRO** or **FSDP** splits that state across GPUs.

A puzzle to start

The model
7.5 billion parameters
(numbers that training learns)

The cluster
64 GPUs
80 GB of memory each

Train it with Adam; every GPU keeps a full copy. Does it fit?

Answer

No. Each GPU needs $\approx$ **120 GB** of training state — more GPUs do not help. Split 64 ways, it is $\approx$ **1.9 GB** per GPU.

Source: Rajbhandari, Rasley, Ruwase & He, "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models," SC 2020.

Recap: the all-reduce

- **all-reduce**: each GPU starts with its own vector; every GPU ends with the **sum**.
- $=$ **reduce-scatter** (GPU k gets the sum of slice k) $+$ **all-gather** (everyone collects all slices).

$$T(S,\ N) = 2(N-1)\ \alpha + 2(N-1)/N \cdot S\ /\ \beta$$

Week 11

α: latency per step, β: bandwidth. Large S: **T** $\approx$ **2S/**β. Small S: the **latency** term wins.

Part 1

Data parallelism

Why is averaging gradients the right thing to do?

Replicate the model, split the batch

global batch, B = 64 examples

shard 0 · 16 examples	shard 1 · 16 examples	shard 2 · 16 examples	shard 3 · 16 examples
↓	↓	↓	↓
GPU 0 · θ forward + backward	**GPU 1** · θ forward + backward	**GPU 2** · θ forward + backward	**GPU 3** · θ forward + backward
g[0]	g[1]	g[2]	g[3]

all-reduce: every GPU gets (g[0] + g[1] + g[2] + g[3]) / 4

every GPU: $\theta \leftarrow$ optimizer step(θ, averaged gradient) $\rightarrow$ copies stay identical

The gradient of a mean is a mean

$$L(\theta) = (1/B)\ \Sigma\ \ell(x;\ \theta) \qquad \text{sum over the global batch}$$

$$g[k] = (1/b)\ \Sigma\ \nabla\ell(x;\ \theta) \qquad \text{sum over shard } k\text{, size } b$$

$$\nabla L = (1/B)\ \Sigma\ b \cdot g[k] \qquad \text{group the sum by shard}$$

$$b = B\ /\ N \quad \Rightarrow \quad \nabla L = (1/N)\ \Sigma\ g[k]$$

Result

Equal shards: **all-reduce SUM, then divide by N**. Exact, not an approximation.

Unequal shards: weight by shard size

$$\nabla L = \Sigma\ (b[k]\ /\ B) \cdot g[k], \qquad B = \Sigma\ b[k]$$

Worker 0
9 examples $\rightarrow$ weight 9/17

Worker 1
8 examples $\rightarrow$ weight 8/17

B = 17 (T3's test)

Watch out

The plain average is **wrong** here. Fix: each rank sends **b[k]** · **g[k]**, all-reduce SUM, divide by **B**.

Sum or mean? Know your loss

Loss on each rank	After all-reduce SUM, divide by
mean over its b examples (equal shards)	N (number of ranks)
sum over its examples	B (global batch)

Watch out

A missing or extra factor is a **silent learning-rate change**. Write the single-process loss first, then match it.

Per-GPU batch, global batch

$$B = N \cdot b$$

- **b**: examples per GPU per step. **B**: examples per optimizer step.
- Fix **B**, change N: same gradient, **same** learning rate.
- Fix **b**, add GPUs: B grows — a **different** problem; retune the learning rate.
- Our MLP: B $= 64$ on N $= 4$ GPUs, so b $= 16$.

Data parallelism by hand

```
for x, y in loader:              # local shard
    opt.zero_grad()
    loss = F.cross_entropy(model(x), y)
    loss.backward()              # local g[k]
    for p in model.parameters():
        dist.all_reduce(p.grad) # SUM, in place
        p.grad /= world_size     # average
    opt.step()                   # same on all
```

- Only **three** new lines.
- One all-reduce per tensor.

Watch out

All ranks must call collectives in the **same order** with the **same shapes**, or the job hangs.

Source: PyTorch documentation, torch.distributed.all_reduce (default op = ReduceOp.SUM, in place).

Close, but not bit-for-bit

Cause	Effect	What to do
Floating-point sum order	last bits differ	compare with a tolerance
Dropout	different random masks	turn it off for the check
Sampler padding	repeated examples	check shards cover B once

In our pipeline

T3 compares DP with a single-process reference **within a tolerance** — know why the tolerance is needed.

Check yourself

Two minutes with a neighbour.

- Why may we average per-GPU gradients at all? State the condition.
- Two ranks get 30 and 34 of a 64-example batch. What should each rank send?
- Ranks use a **sum** loss, but you all-reduce and divide by N. By what factor is the gradient off ($B = 64$, $N = 4$)?

Part 2

The cost, and how DDP hides it

How long does agreeing on a gradient take?

Cost for our MLP

$$S = 34{,}338 \times 4 \text{ bytes} = 137{,}352 \text{ bytes}$$

$$\text{bandwidth term: } \; 2 \cdot 3/4 \cdot S \; / \; \beta \; \approx \; 0.46 \; \mu s$$

$$\text{latency term: } \; 2(N-1) \; \alpha \; \approx \; 30 \; \mu s$$

Memory

Our all-reduce is **latency-bound**. Fewer, larger messages help; more bandwidth does not.

Source: H100 datasheet (NVLink 900 GB/s bidirectional); $\alpha \approx 5$ µs assumed — use your T2 fit.

Meet a bigger model

Our MLP

$\Psi = 34{,}338$

A large language model

$\Psi = 7.5 \times 10^9$

Ψ = number of parameters. Today a model is just Ψ numbers.

$$\text{bf16 gradient} = 2 \text{ bytes} \times \Psi = 15 \text{ GB}$$

Definition

bf16: a 2-byte number format GPUs compute with fast. Bytes = $\Psi \times$ bytes per number.

Cost for the 7.5-billion-parameter model

$$S = 15 \text{ GB of bf16 gradients}$$

Where the GPUs are	All-reduce time
8 GPUs in one server (NVLink)	$\approx$ 58 ms
64 GPUs in 8 servers (network)	$\approx$ 0.59 s

Memory

Large models are **bandwidth-bound**: $T \approx 2S/\beta$ per step, and crossing servers costs $\approx 10\times$.

Source: NVIDIA DGX H100 User Guide (8× ConnectX-7, 400 Gb/s per GPU); H100 datasheet (NVLink 900 GB/s bidirectional).

DDP: the same loop, wrapped

```
dist.init_process_group("nccl")
torch.cuda.set_device(local_rank)  # Week 10
model = DDP(MLP().cuda(), device_ids=[local_rank])
opt = torch.optim.AdamW(model.parameters())
for x, y in loader:       # this rank's shard
    opt.zero_grad()
    F.cross_entropy(model(x), y).backward()
    opt.step()
```

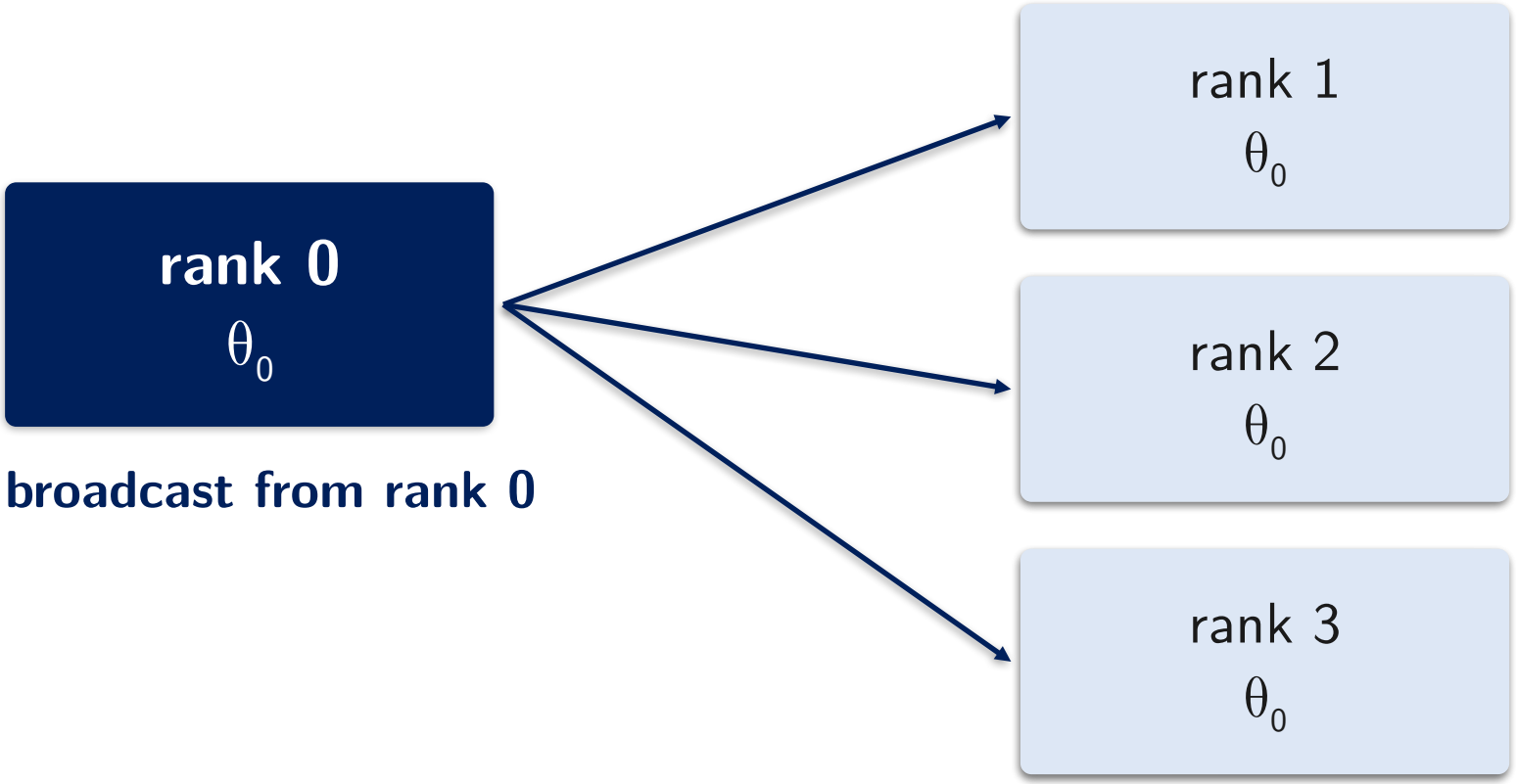
- Same loop as on one GPU.
- **backward()** also averages the gradients.

Key idea

DDP does three things: **start identical**, **bucket** the gradients, **overlap** the all-reduce with backward.

Source: PyTorch documentation, torch.nn.parallel.DistributedDataParallel and "DDP notes"; Li et al., VLDB 2020.

Start identical, stay identical



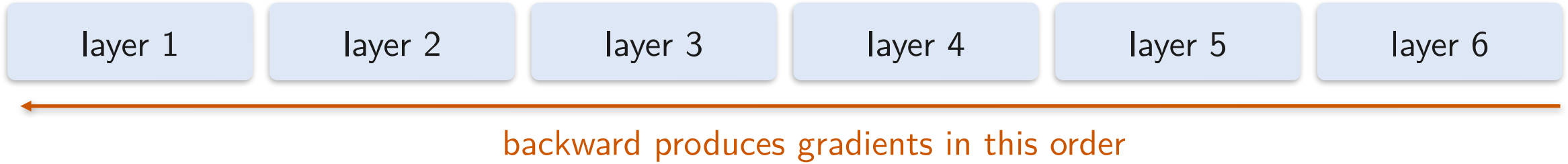
WHY IT LASTS

Same start + same averaged gradient + same optimizer $\Rightarrow$ the copies **stay identical**.

Source: PyTorch documentation, torch.nn.parallel.DistributedDataParallel and "DDP notes"; Li et al., VLDB 2020.

Buckets: fewer, larger all-reduces

a model's layers, input to output





one all-reduce per bucket, as soon as all of its gradients are ready

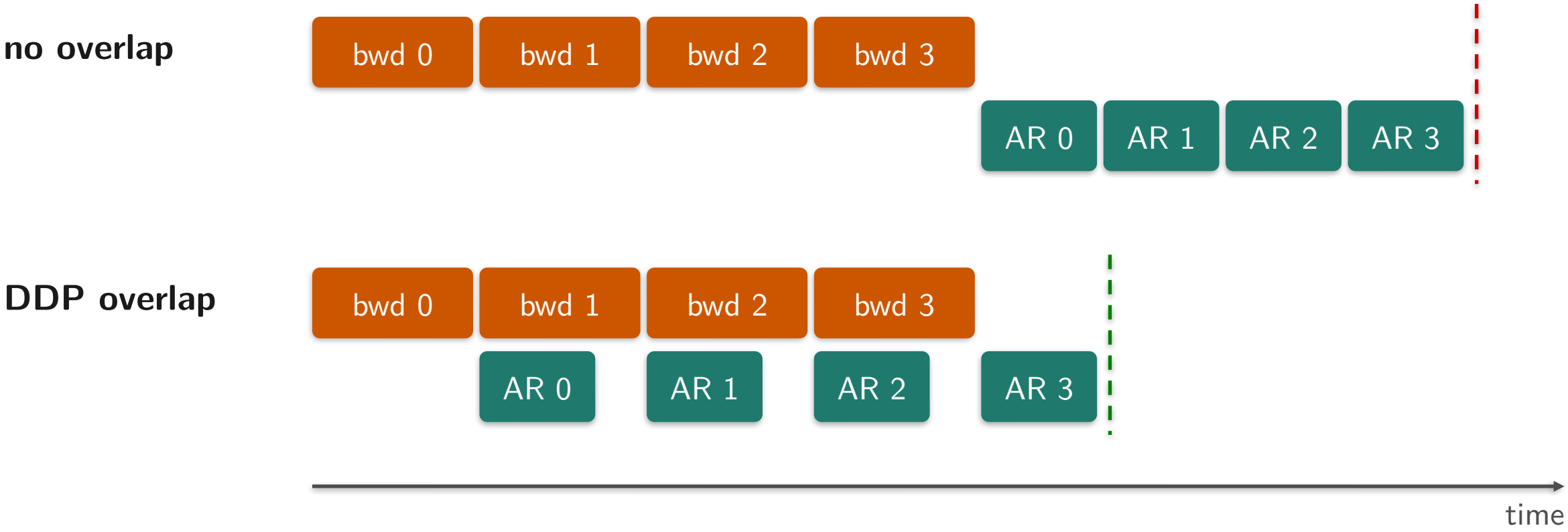
Memory

Fewer calls means fewer latency costs — and bucket 0 can be sent **while backward continues**.

Source: PyTorch documentation, torch.nn.parallel.DistributedDataParallel and "DDP notes"; Li et al., VLDB 2020.

Overlap: send while backward runs

bwd: backward of one bucket · AR: its all-reduce



Key idea

Step time $\approx$ max(backward, all-reduce) + **one bucket**, instead of the sum.

Check yourself

- Which term of the ring formula dominates for our MLP's gradient? For the 15 GB one?
- Why does DDP start by copying rank 0's parameters to every rank?
- Will DDP's overlap speed up our 34,338-parameter MLP? Why or why not?

Part 3

Memory: ZeRO and FSDP

Every GPU holds the same 16 bytes per parameter. Why?

Where the memory goes: 16 bytes per parameter

16-bit parameters	2Ψ
16-bit gradients	2Ψ
FP32 master parameters	4Ψ
Adam: average gradient	4Ψ
Adam: average squared gradient	4Ψ

optimizer state: 12Ψ

$$\text{total} = (2 + 2 + 12)\ \Psi = 16\Psi \text{ bytes}$$

$\Psi = 7.5 \times 10^9$: **120 GB**

our MLP: $\approx$ **0.55 MB**

Source: Rajbhandari, Rasley, Ruwase & He, "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models," SC 2020. Section 3.1.

Data parallelism copies all of it



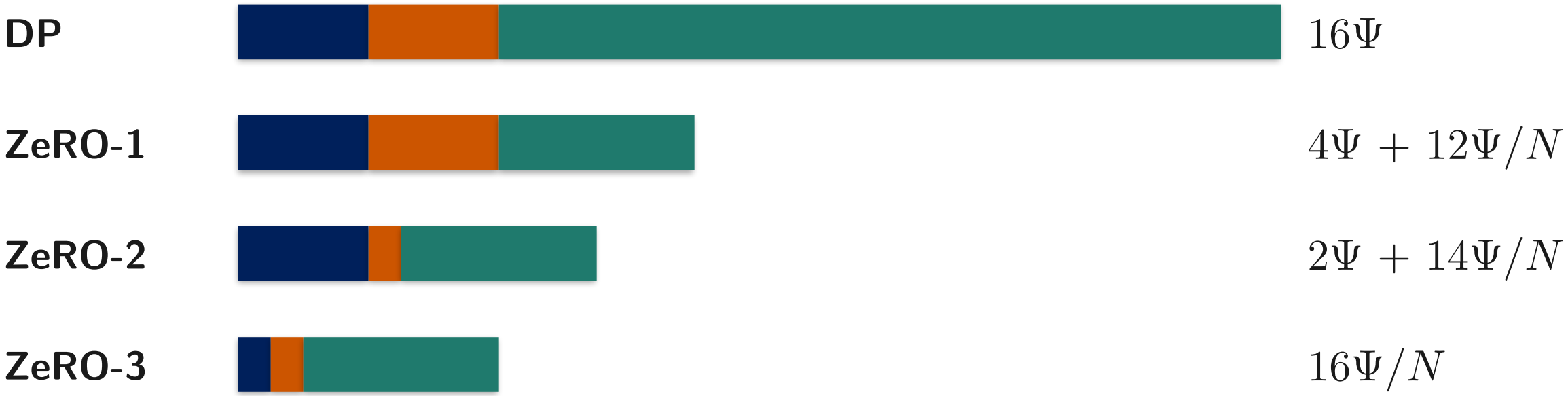
blue: 16-bit params 2Ψ · orange: 16-bit grads 2Ψ · green: optimizer 12Ψ

Key idea

N identical copies of the optimizer state — **75%** of the memory — that each GPU could own one N-th of.

ZeRO: each GPU keeps a 1/N slice

shard = cut a state into N slices; GPU k keeps slice k (one GPU of N = 4, to scale)



Memory

Only **stage 3** keeps shrinking as N grows.

Source: Rajbhandari, Rasley, Ruwase & He, "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models," SC 2020. Figure 1.

Back to the puzzle

$\Psi = 7.5 \times 10^9$ parameters, N = 64 GPUs of 80 GB

Method	Model state per GPU	Fits?
Plain data parallelism	120 GB	no
ZeRO-2	16.6 GB	yes
ZeRO-3	1.9 GB	yes

Answer

Sharding turns an impossible job into one that fits — the GPUs stop storing **copies**.

Source: Rajbhandari, Rasley, Ruwase & He, "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models," SC 2020. Figure 1.

What sharding costs in communication

Method	What moves each step	Per GPU
DP, ZeRO-1, ZeRO-2	gradients summed + parameters gathered	2Ψ
ZeRO-3	parameters gathered **twice** + gradients summed	3Ψ

Check

Stages 1–2 save memory **for free**. Stage 3 divides everything by N for **+50%** traffic.

Source: Rajbhandari, Rasley, Ruwase & He, "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models," SC 2020. Section 7.2.

FSDP: PyTorch's ZeRO-3

forward



free after use

backward



units 0, 1, 2 = embedding, hidden layer, output layer

AG: all-gather a unit's parameters · RS: reduce-scatter its gradients

Key idea

Gather a unit **just before** it is used, free it **right after**: only about one unit is whole at a time.

Source: PyTorch documentation, torch.distributed.fsdp.fully_shard (FSDP2); Zhao et al., "PyTorch FSDP," VLDB 2023.

Check yourself

- List the five pieces of the 16 bytes per parameter.
- ZeRO-2, $N = 8$, $\Psi = 1$ billion: how many GB of model state per GPU?
- Why does stage 3 move $1.5\times$ the data of plain data parallelism?

IN-CLASS BUILD

Check an update, count memory

Work in pairs. Part A is the core of T3; Part B is pencil and paper.

Exercise A: check a data-parallel update

```
xb, yb = shard(X, Y, rank, world)  # disjoint
F.cross_entropy(model(xb), yb).backward()
for p in model.parameters():
    dist.all_reduce(p.grad)        # SUM
    p.grad /= ____                 # (1)
F.cross_entropy(ref(X), Y).backward()  # 1 proc
torch.testing.assert_close(grads(model),
                           grads(ref))
```

QUESTIONS

(1) Fill in the blank.

(2) 17 examples on 2 ranks, 9 and 8: what changes?

(3) Use DDP instead. Does it still pass with 9 and 8?

model and ref start with the same weights; ref sees the whole batch on one process.

Exercise B: account for memory

DP: $16\Psi = ___$ GB

ZeRO-1: $4\Psi + 12\Psi/N = ___$ GB

ZeRO-2: $2\Psi + 14\Psi/N = ___$ GB

ZeRO-3: $16\Psi/N = ___$ GB

GIVEN

$\Psi = 13 \times 10^9$, N = 8 GPUs, at most 60 GB of model state per GPU. GB = 10^9 bytes.

QUESTION

Which stages fit? Which would you pick, and why?

Solutions A: the update check

- (1) `world` — the number of ranks (equal shards).
- (2) Each rank sends b[k] $\cdot$ g[k]; after the SUM, divide by B $= 17$.
- (3) DDP divides by N: it **fails** for $9 + 8$ unless you rescale each rank's loss.

In our pipeline

Passing on $8 + 8$ is not enough: T3 tests the **uneven** case.

Solutions B: the memory count

DP: $16\Psi = 208$ GB

ZeRO-1: $4\Psi + 12\Psi/N = 71.5$ GB

ZeRO-2: $2\Psi + 14\Psi/N = 48.75$ GB

ZeRO-3: $16\Psi/N = 26$ GB

Check

ZeRO-2 is the cheapest that fits: under 60 GB at DP's communication.

How to test it

- Run on **CPU** first (gloo backend), then on GPUs (NCCL).
- Compare **gradients and parameters** after the optimizer step.
- Use `assert_close` with its **default** tolerances.
- Test an **uneven** batch of 17.
- Delete the division: the test must **fail**.

In our pipeline

T3, part 1: hand-written DP vs a single-process reference vs DDP, on D3 data.

Key takeaways

- Gradient of a mean = mean of gradients: **all-reduce SUM, divide by N** — equal shards only; otherwise weight by **b[k]/B**.
- One all-reduce per step: **latency-bound** for our MLP, $\approx$ **2S**/β for big models.
- **DDP**: start identical, bucket the gradients, overlap the all-reduce with backward.
- Training with Adam: **16 bytes per parameter**, copied on every GPU by plain DP.
- **ZeRO-1/2** shard it at DP's traffic; **ZeRO-3 / FSDP** shard everything for 1.5×.

Next time, and readings

When	What
Friday	T2 due (distributed programming)
After this lecture	T3 released: DP vs single-process reference vs DDP, on D3 data
Lecture 2 (this week)	Tensor and model parallelism: splitting one layer across GPUs

READING

Rajbhandari et al., "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models," SC 2020 — Sections 1, 3, 5 and 7.